MPTA-Repair: Clock-Bound Repair for Multi-Property Timed Automata in Industrial Practice

***Abstract*—Timed automata (TAs) are widely used to model and verify real-time industrial systems, which are typically required to satisfy multiple predefined properties simultaneously. When verification fails, repairing the model is challenging because it requires modifying erroneous clock constraints to satisfy all properties while preserving functional equivalence. To address this, we present MPTA-Repair, an automated repair framework for multi-property TAs. The framework employs an iterative, counterexample-guided MaxSMT approach to generate minimally modified candidate models. To optimize search efficiency, it systematically exploits relationships among properties, counterexamples, and repair variables. Each candidate model is then validated through full-property regression verification and acceptability checks. Compared to the TARTAR baseline, MPTA-Repair achieves higher effectiveness on faulty models generated via our proposed mutation strategy. Specifically, it improves the success rate from 72% to 93% on benchmarks derived from UPPAAL and the XtaBenchmarkSuite, and from 20% to 76% on an industrial dataset constructed in this work.**

***Index Terms*—Timed automata, automated model repair, MaxSMT, multi-property verification**

I. INTRODUCTION

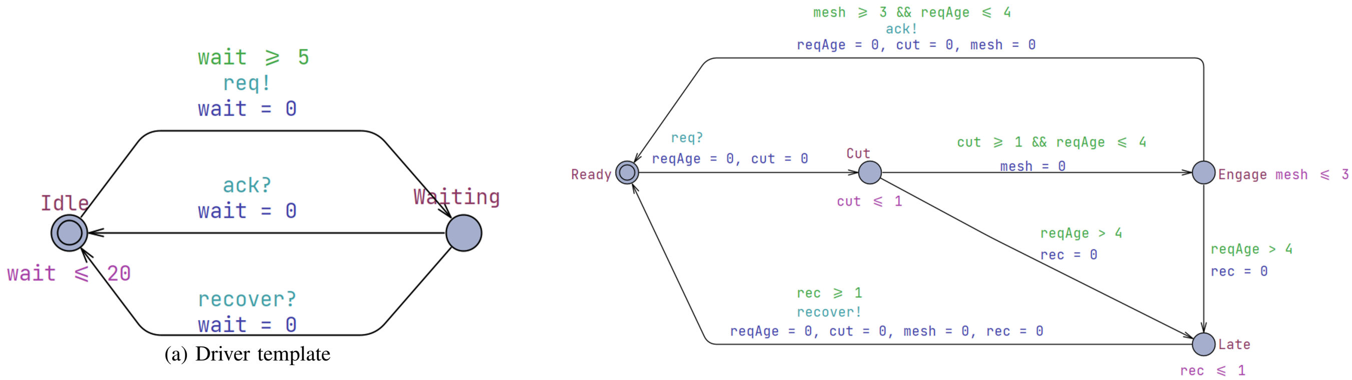
Timed automata (TAs) are widely used to model and verify real-time industrial systems whose correctness depends on both discrete behavior and quantitative timing constraints [1], [2]. Such systems commonly appear in railway control and communication protocols [3], automotive and autonomous-driving logic [4], [5], medical cyber-physical systems [6], [7], and aerospace or avionics scheduling [8]. In these domains, an alarm, pulse, message, or resource release may be functionally present but still incorrect if it occurs too early, too late, or for an incorrect duration. Verification therefore has to check not only reachability or event order, but also whether timing requirements are satisfied under all relevant executions.

Industrial timed-automata models are rarely validated against a single property. They are typically checked against a set of predefined properties that jointly express safety, bounded response, deadline satisfaction, error-state reachability, alarm duration, and deadlock freedom [9]. When a property is violated, model checkers such as UPPAAL, Kronos, or opaal can produce a timed diagnostic trace (TDT), namely a timed counterexample showing how the model reaches a violating state [10]. A TDT is useful evidence for repair, but it is not a repair instruction. It does not identify which clock constraint is erroneous, whether the error lies in a transition guard or a location invariant, or how a numerical bound should be changed to restore the intended timing behavior [11].

The difficulty becomes more pronounced in the multi-property setting. A timing fault may affect several properties, and different violated properties may produce related counterexamples. A repair that blocks one reported trace may leave another violating trace feasible, or may break a property that was previously satisfied. Similarly, changing a bound for one property can affect another property when the same clock, location, transition, synchronization, or execution fragment is involved. Thus, a practical repair method for industrial timed automata must not accept a candidate merely because it fixes one local counterexample. It must validate the candidate against the complete property set and ensure that the functional behavior of the model is preserved.

Figure 1 illustrates this issue with a simplified network timed automaton for an automotive gear-shift controller [4]. A Driver template periodically issues a shift request, and a GearController template performs torque cut, gear engagement, acknowledgement, and recovery. The model uses $cut \leq 1$ in the Cut location, $cut \geq 1$ on the transition to Engage, $mesh \leq 3$ in the Engage location, and $mesh \geq 3$ on the acknowledgement transition. One property requires at least two time units of torque cut before gear engagement, while another property requires the shift request to be acknowledged within four time units. A single-property repair for the first property may change only the cut bounds from 1 to 2 and thereby block the early-engagement trace. However, with the mesh bounds unchanged at 3, the earliest acknowledgement would occur at time 5, so the response-deadline property fails. A multi-property repair instead changes both the cut bounds and the mesh bounds, for example to $cut \leq 2$, $cut \geq 2$, $mesh \leq 2$, and $mesh \geq 2$. The repaired controller then satisfies both the minimum torque-cut requirement and the response deadline while preserving the same untimed request-acknowledgement behavior. This example motivates treating properties, counterexamples, and repair variables as joint repair evidence rather than independent local repair tasks.

Existing repair techniques provide an important foundation for this problem. Clock-bound repair encodes a TDT as linear real arithmetic, introduces variation variables for clock bounds, and uses MaxSMT solving to compute small modifications to guards and invariants [11]. TARTAR builds on this idea by suggesting syntactic repairs and checking whether the repaired model preserves the untimed functional behavior of the original network timed automaton [12]. Related work also includes parameter synthesis for parametric timed automata



(a) Driver template

(b) GearController template

Fig. 1. A simplified network timed automaton for a cyclic automotive gear-shift controller. The faulty model uses $cut \leq 1$, $cut \geq 1$, $mesh \leq 3$, and $mesh \geq 3$. A repair that changes only the cut bounds blocks the early-engagement trace but can violate the response-deadline property; a multi-property repair also changes the mesh bounds.

[13], clock-guard repair through abstraction and testing [14], robustness analysis for uncertain timing constants [15], and SAT/SMT-based model or program repair [16]. These methods show that automated repair of timed models is feasible, but most TDT-based workflows still reason from one violated property or one diagnostic trace at a time.

This paper presents **MPTA-Repair**, an automated repair framework for multi-property TAs. Given a faulty timed-automata model, a set of timing properties, and diagnostic traces produced by verification, MPTA-Repair searches for small changes to numerical clock bounds in transition guards and location invariants. It does not change property formulas, locations, transitions, synchronization labels, clock references, or resets. This restriction matches the clock-bound repair formulation and the mutation-based evaluation used in this paper, while broader modeling faults such as reset errors, synchronization errors, data-update errors, or structural errors are outside the current scope.

MPTA-Repair employs an iterative, counterexample-guided MaxSMT approach to generate minimally modified candidate models. Instead of treating violated properties and diagnostic traces as independent subproblems, it maintains a pool of property-trace repair obligations and searches for a clock-bound assignment that satisfies the currently encoded obligations. To improve search efficiency, the framework systematically exploits relationships among properties, counterexamples, and repair variables to organize the search. These relations group related evidence, rank or restrict active repair variables, and avoid solving over unrelated bounds when the current counterexample evidence gives no reason to include them. These relations guide candidate generation, whereas accepted candidates must still satisfy all properties in regression verification and pass a TARTAR-style acceptability check based on functional equivalence.

We evaluate MPTA-Repair on faulty models generated via a mutation strategy designed for clock-bound repair. The benchmark dataset is derived from UPPAAL and the XtaBenchmarkSuite, while the industrial dataset is constructed in this work from practical case studies covering railway alarm and communication scenarios [3], [17], train-gate behavior [18], automotive gear control [4], cardiac pacing [6], autonomous road-junction rules [5], mechanical ventilation [7], and aerospace scheduling [8]. We measure repair effectiveness, repair size, runtime, regression outcomes, and behavior preservation. Compared with an iterative TARTAR-style baseline under the same repair space, full-property regression requirement, and admissibility requirement, MPTA-Repair improves the repair success rate from 72% to 93% on the benchmark dataset and from 20% to 76% on the industrial dataset.

The contributions of this paper are as follows. First, we formulate clock-bound repair for practical timed-automata models as a multi-property, multi-counterexample repair problem in which local trace repair is insufficient unless the repaired model satisfies the complete property set. Second, we present MPTA-Repair, an iterative counterexample-guided MaxSMT framework that uses relationships among properties, counterexamples, and repair variables to guide the repair search. Third, we implement a prototype that combines trace analysis, MaxSMT-based clock-bound repair, full-property regression verification, and TARTAR-style admissibility checking. Fourth, we evaluate the approach on benchmark and industrial datasets against an iterative single-property baseline under identical acceptance criteria. Finally, we report practical lessons on property selection, repair interpretability, counterexample accumulation, and the need for full regression after each candidate repair.

The remainder of this paper is organized as follows. Section II provides the background and problem setting, including timed automata, diagnostic traces, clock-bound repair, and the multi-property repair objective. Section III presents MPTA-Repair and its relation-guided repair workflow. Section IV presents the implementation and experimental evaluation, including datasets, results, and practical lessons. Section V concludes the paper.

II. Background and Motivation

A. Timed Automata and Industrial Networks

Timed automata provide a standard formal model for systems whose correctness depends on both discrete control decisions and quantitative time constraints [1], [2]. Let C be a finite set of clocks. A clock constraint over C is a conjunction of atomic constraints of the form $x \sim b$, where $x \in C$, $\sim \in \{<, \leq, =, \geq, >\}$, and b is a non-negative numerical bound. We write $\mathcal{B}(C)$ for the set of such constraints. A timed automaton is a tuple

$$T = (L, \ell_0, C, \Sigma, \Theta, I),$$

where L is a finite set of locations, $\ell_0 \in L$ is the initial location, Σ is a set of action labels, Θ is a finite set of transitions, and $I : L \to \mathcal{B}(C)$ assigns an invariant to each location. A transition $\theta \in \Theta$ is written as

$$\theta = (\ell, g, a, R, \ell'),$$

where ℓ and ℓ' are the source and target locations, $g \in \mathcal{B}(C)$ is the guard, $a \in \Sigma$ is the action label, and $R \subseteq C$ is the set of clocks reset by the transition.

The semantics is defined over states (ℓ, u), where ℓ is a location and $u : C \to \mathbb{R}_{\geq 0}$ is a clock valuation. A delay step lets time elapse in the current location as long as the location invariant remains satisfied. A discrete step can be taken when the current clock valuation satisfies the transition guard; the step moves the automaton to the target location and resets the clocks in R. Thus, guards constrain when discrete actions are enabled, whereas invariants constrain how long the automaton may remain in a location.

Industrial timed-automata models are usually networks rather than single automata. A network

$$\mathcal{N} = T_1 \parallel \cdots \parallel T_k$$

composes templates for controllers, plants, environments, communication protocols, schedulers, and monitors [19]. The exact product semantics depends on the modeling language and tool, for example on binary synchronization, broadcast channels, urgent or committed locations, and data variables. In this work these structural and data-level features are treated as fixed modeling context. The repair space is deliberately restricted to numerical clock-bound occurrences in guards and invariants; synchronization labels, reset sets, clock references, locations, transitions, data updates, and property formulas are not repair variables.

This restriction matches a common class of industrial modeling faults. Timing constants often represent design thresholds, sampling periods, timeout values, alarm durations, refractory intervals, or scheduling deadlines. For example, vehicle controllers may need to complete a gear-shifting sequence within a bounded response time [4]; railway models may encode gate, alarm, or communication timeout requirements [3]; medical devices may encode physiological timing windows [6]; and avionics or aerospace models may encode execution-time bounds and deadlines [20]. In such models, an incorrect numerical bound can make an otherwise intended behavior occur too early, too late, or for an invalid duration.

We do not assume that all modeling errors are clock-bound errors. Industrial models may also contain wrong resets, missing transitions, incorrect synchronizations, erroneous data updates, or incomplete properties. These faults are outside the current repair space. The assumption in this paper is narrower: the discrete model structure is treated as the intended design, while one or more numerical timing bounds may be inaccurate due to calibration, transcription, dimensioning, or design-space uncertainty. This is the fault model targeted by clock-bound repair [11] and is consistent with mutation-based repair and testing, where faults are modeled as small deviations from an otherwise near-correct artifact [21].

B. Properties and Timed Diagnostic Traces

Timed-automata models are checked against timing properties using real-time model checkers such as UPPAAL [19]. This paper focuses on safety-style verification obligations. Direct examples include queries of the form $A[]\,\psi$, where every reachable state must satisfy the state predicate ψ, and deadlock-freedom checks such as $A[]\ not\ deadlock$. Bounded-response and deadline requirements are handled when they are encoded by observer or monitor templates into safety-style violations, for example by reaching an error or late location [9]. Therefore, the repair procedure reasons about property violations that can be represented as reachable safety violations in the instrumented timed-automata network.

A discrete trace skeleton is a finite sequence of transitions

$$\tau = \theta_0, \theta_1, \ldots, \theta_{n-1}.$$

A timed realization of τ is a sequence of non-negative delays and clock valuations that makes the transition sequence executable from the initial state while respecting guards, resets, invariants, and network synchronization. The skeleton τ is feasible if it has at least one timed realization. Given a property φ, a feasible skeleton is a timed diagnostic trace (TDT) for φ if at least one of its timed realizations reaches a state that violates the safety condition associated with φ [11]. Depending on the model checker, the reported counterexample may contain concrete delays, symbolic zones, or only enough information to reconstruct the discrete path and timing constraints.

A TDT is evidence of failure, not a root-cause explanation. It shows that some timing realization of some discrete behavior violates a property, but it does not identify which numerical bound is wrong, whether the fault lies in a guard or invariant, or how the model should be changed. Several bounds may jointly enable the violating realization, and one incorrect bound may contribute to several diagnostic traces. Conversely, blocking the discrete path of a TDT is usually not a valid repair: it may remove intended functional behavior, such as a request, acknowledgement, actuation, communication step, or scheduling decision. A useful repair should change the timing envelope of the behavior while preserving the relevant untimed functionality of the model.

C. Clock-Bound Repair and Admissibility

A repairable site is an occurrence of a numerical bound b in an atomic clock constraint $x \sim b$ that appears in a transition guard or a location invariant. A clock-bound repair changes this occurrence to a new value b', equivalently by introducing a variation variable v and using the repaired bound $b + v$. The comparison operator, clock reference, transition structure, location structure, synchronization label, and reset set are kept fixed. Bounds are constrained to remain in the numerical domain supported by the target model representation.

TDT-based clock-bound repair encodes the feasibility of a diagnostic trace as constraints over delays and clock values, then introduces variation variables for the bounds that occur in the encoded guards and invariants [11]. MaxSMT solving can then be used to prefer small repairs, for example by maximizing soft constraints of the form $v = 0$ and, when needed, minimizing the magnitude of nonzero variations [22]. In the single-trace setting, the resulting candidate is local: the same discrete trace skeleton should remain feasible, but no feasible realization of that skeleton should still witness the considered property violation.

Local trace repair is not sufficient for model repair. A bound change that eliminates one violating realization may create a new violation on another execution, fail another property, or make intended functionality unavailable. MPTA-Repair therefore treats MaxSMT-based trace repair as candidate generation only. A candidate is accepted only after two global checks. First, the repaired model must satisfy the complete property set under regression verification. Second, it must pass an admissibility check based on TARTAR-style functional equivalence [11].

The admissibility check compares untimed functional behavior rather than timed language equality. This distinction is necessary because a successful timing repair is expected to remove some violating timed realizations. Let Σ_f denote the functional alphabet used for admissibility, after excluding monitor-only, observer-only, and internal labels when appropriate. The intended criterion is

$$\mathrm{Untimed}_{\Sigma_f}(M_{bad}) = \mathrm{Untimed}_{\Sigma_f}(M_{rep}),$$

meaning that the repaired model should not add or remove functional action traces, although the timing of those traces may change. This criterion prevents repairs that satisfy safety properties merely by disabling intended behavior.

D. Why Multi-Property Repair Is Needed

Industrial timed-automata models are normally validated by a set of properties rather than by a single assertion [9]. The properties jointly describe the reliability envelope of the model, including absence of unsafe states, bounded response, timeout handling, deadline satisfaction, admissible residence times, and deadlock freedom. Let M_{bad} be a faulty timed-automata network and let

$$\Phi = \{\varphi_1, \ldots, \varphi_m\}$$

be the property set used to validate it. The goal is to construct a repaired model M_{rep} by changing a small number of clock-bound constants in guards and invariants such that:

$$M_{rep} \models \Phi,$$

the repaired bounds are well formed, and the repaired model preserves the projected untimed functional behavior of M_{bad}.

The need for joint reasoning can be seen in the gear-shift controller used as a running example. The driver periodically issues a shift request, and the controller performs torque cut, gear engagement, acknowledgement, and recovery. Suppose the faulty controller uses bounds such as $cut \le 1$, $cut \ge 1$, $mesh \le 3$, and $mesh \ge 3$. One property may require at least two time units of torque cut before gear engagement, while another property may require acknowledgement within four time units of the request. A single-property repair that changes only the cut bounds from 1 to 2 can block the early-engagement violation. However, if the $mesh$ bounds remain at 3, the earliest acknowledgement may occur at time 5, thereby violating the response deadline. A multi-property repair must instead consider both requirements and may need to change both the cut and $mesh$ timing bounds while preserving the same untimed request–engage–acknowledge behavior.

This example illustrates why violated properties, diagnostic traces, and repair variables should not be treated as unrelated subproblems. Different properties may share clocks, locations, synchronizations, or execution fragments. Multiple TDTs may expose different realizations of the same underlying timing defect. Conversely, a repair computed from one trace can be locally valid yet globally unacceptable after full regression. MPTA-Repair addresses this setting by accumulating multi-property and multi-counterexample repair evidence, generating small clock-bound candidates with MaxSMT, and accepting a candidate only after complete property regression and functional-admissibility checking.

III. Approach

A. Overview

MPTA-Repair is a counterexample-guided repair workflow for multi-property, multi-counterexample clock-bound repair of industrial timed-automata models [23]. Its input is a faulty model `M`, a property set Φ = `{φ1, ..., φm}`, and configuration parameters such as the maximum number of refinement rounds, candidate limits, and a global timeout. The repairable bounds are not given as a separate user input; they are extracted from the numerical clock bounds occurring in transition guards and location invariants [11]. The output is either a repaired model `M'` together with a set of clock-bound edits, or a failure report explaining why no admissible full-property repair was found within the configured budget.

The workflow is designed around two principles. First, candidate repair generation may be driven by local diagnostic traces, but candidate acceptance must be global. A candidate that repairs one violated property may cause another property to fail, because the same clock, location, transition, or synchronization may participate in several timing requirements. MPTA-Repair therefore keeps collecting counterexamples from newly violated properties and uses them to further restrict the set of admissible repair candidates. This refinement continues until a candidate satisfies the complete property set and passes admissibility checking, or until the configured budget is exhausted. In our evaluation, we use a fixed timeout, for example 10 minutes per repair instance, to avoid unbounded refinement [24].

Second, multiple properties and multiple diagnostic traces are not treated as unrelated repair subproblems. A single faulty timing bound may contribute to several violations, and several traces may expose different timing realizations of the same underlying defect. MPTA-Repair therefore maintains a pool of property-trace obligations and searches for a clock-bound assignment that satisfies all currently encoded obligations. Relation analysis among properties, traces, and repairable bounds is used to reduce redundant work and guide the search, but it is not used as a substitute for verification [23].

At a high level, MPTA-Repair proceeds as follows. It verifies the model against all properties, collects TDTs for violated properties, extracts repairable clock-bound values, and builds a joint MaxSMT repair problem from the current property-trace pool. The resulting assignment is instantiated into a candidate model. The candidate is then checked against the complete property set and against a TARTAR-style admissibility criterion [11]. If validation fails, the rejected assignment is blocked, newly observed diagnostic traces are added, and the repair loop continues.

Figure 2 summarizes the resulting workflow. The initial verification stage produces the property-trace pool, the relation-guided stage organizes properties, counterexamples, and repairable bounds, and the MaxSMT stage generates candidate edits. The final validation gates either accept a fully admissible repair or feed new diagnostic evidence back into the next refinement round.

B. Bound Variation and Joint Trace Encoding

MPTA-Repair follows the bound-variation view used in clock-bound repair [11]. Each editable numerical bound in a guard or invariant is associated with a variation variable, so a repaired constraint has the same clock reference and comparison operator as the original constraint but a shifted numerical value. The current repair pipeline therefore changes only clock-bound constants. Transition structure, location structure, synchronization labels, reset sets, and discrete updates are outside the main repair space.

For each diagnostic trace, MPTA-Repair constructs a TARTAR-style trace constraint system [11]. The encoding contains the standard conditions for clock initialization, time elapse, clock resets, guard satisfaction, and invariant satisfaction along the observed trace. The only difference from the original trace constraints is that repairable bounds are replaced by their varied values. This keeps the discrete trace skeleton visible to the solver while allowing the timing region of that skeleton to change.

For each violated property and diagnostic trace, the repair obligation has two roles. First, it preserves the executability of the same discrete trace skeleton after repair, so the method does not remove a functional path merely to avoid the violation. Second, it requires that no feasible timing realization of that skeleton still violates the considered property. In this way, the local obligation blocks the violating timed behavior rather than blocking the underlying functional behavior.

MPTA-Repair builds one joint MaxSMT problem from the current pool of property-trace obligations. The hard part of the problem includes all encoded trace obligations, well-formedness constraints for repaired bounds, and blocking constraints for candidates that have already failed validation. The soft part prefers unchanged bounds. Maximizing these soft constraints minimizes the number of edited clock bounds [22]; when needed, a secondary objective minimizes the total magnitude of the nonzero edits. If a candidate is rejected by full-property regression or admissibility, the refinement loop records a blocking constraint so that the same assignment is not returned again. This generalizes single-trace repair from one diagnostic trace to a pool of currently relevant property-trace obligations.

C. Relation-Guided Search Optimization

As refinement proceeds, the number of violated properties, diagnostic traces, and editable clock bounds may grow [23]. Encoding every possible obligation and every editable bound without organization can make the MaxSMT problem larger and may lead to repeated solving over redundant evidence. MPTA-Repair therefore uses relation analysis as a search optimization. These relations guide pruning, grouping, and ranking, but every accepted repair is still validated by full-property regression and admissibility.

At the property layer, MPTA-Repair handles safety properties in a practical normalized fragment. A property is normalized into an `A[]`-style condition over location predicates and clock or integer comparisons [9]. The relation analysis uses conservative syntactic and formula-level rules. Two properties are treated as equivalent only when their normalized forms are identical. Dominance is used only when it can be established safely, such as when two upper-bound properties differ only in the numerical threshold and one threshold is strictly stronger than the other. Similar rules apply only when the target predicate, clock, and operator pattern match after normalization. Shared clocks, target locations, and templates are recorded as dependency information, but they are not by themselves used to remove obligations.

At the counterexample layer, MPTA-Repair maintains a pool of TDTs associated with violated properties. Exact duplicates are removed, and traces are annotated with their involved transitions, locations, clocks, guard bounds, invariant bounds, and source properties [11]. Structural similarity, shared prefixes, and shared repair variables are used to organize the pool and explain why certain traces should be considered together. However, a trace is removed from the active obligation set only when it is an exact duplicate or when a sound subsumption

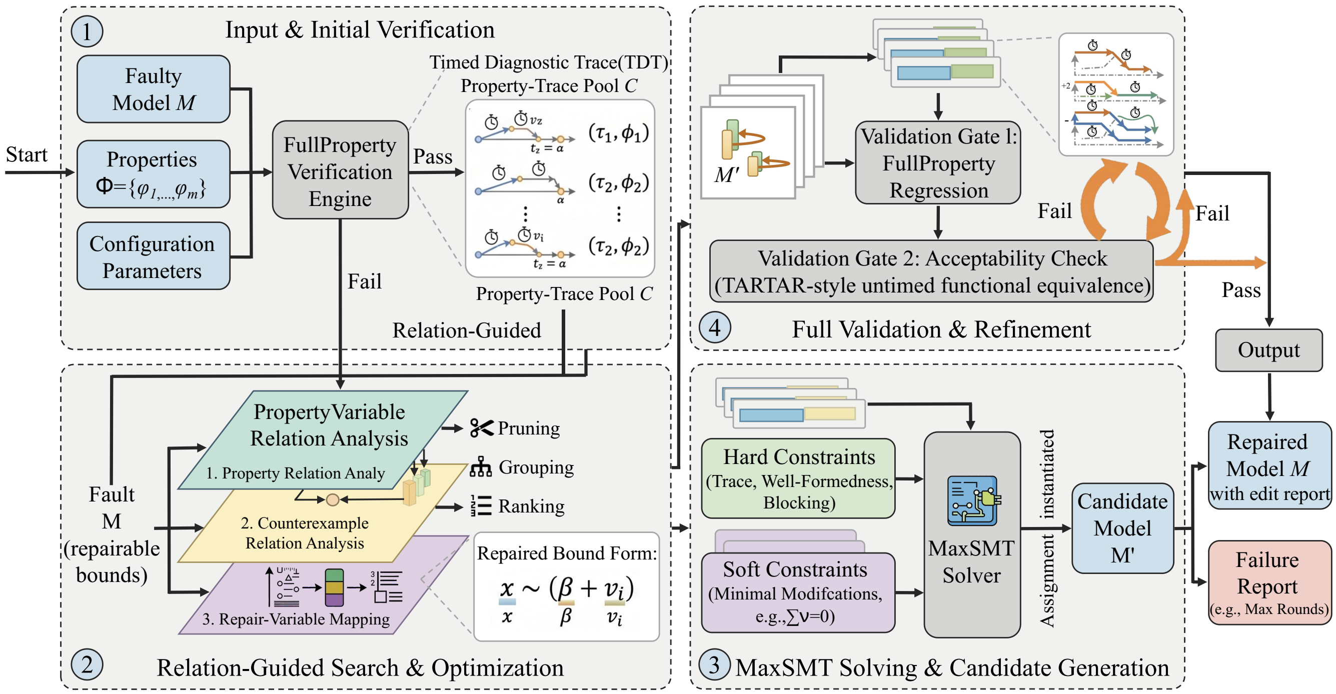


Fig. 2. Overview of the MPTA-Repair workflow.

check shows that its local obligation is covered by another encoded obligation. Otherwise, the trace remains available for joint encoding.

At the repair-bound layer, MPTA-Repair maps each property-trace obligation to the clock-bound values that can affect it. This mapping helps restrict the active repair variables to those appearing in or influencing the current counterexample pool. It also helps order candidate generation when several bounds are available. However, relation scores do not prove correctness, and they do not force the solver to modify a particular bound. They only reduce the search effort by avoiding unrelated repair variables when the current counterexample evidence gives no reason to open them.

D. Optimization Objective

The MaxSMT objective is intentionally simple [25]. The primary objective is to minimize the number of changed clock bounds by maximizing soft constraints of the form `v = 0`. Among candidates with the same number of changed bounds, the implementation may minimize the total magnitude of the changes. This follows the engineering goal of producing small and inspectable repairs.

MPTA-Repair does not optimize directly for speculative notions such as semantic risk or causal relevance. Relation analysis is used before and around solving to select or rank repair variables and obligations. The solver objective itself remains centered on minimality of the edit set and, when enabled, magnitude of the edit values.

Static consistency constraints are enforced during candidate generation [26]. Repaired bounds must remain in the supported numerical domain. When a lower and an upper bound jointly constrain the same clock in a local region, the repaired values must not make the timing interval inconsistent. Since operators and clock references are fixed in the current repair space, generated assignments change only numerical bound values [14].

E. Validation, Admissibility, and Refinement

Candidate validation is the main correctness boundary of MPTA-Repair [27]. After applying a candidate edit set, the repaired model is verified against every property in Φ. A candidate that repairs the triggering property but violates another property is rejected. Such a failure means that the current encoded trace pool was insufficient: the candidate satisfied all encoded local obligations, but the repaired model still has an execution that violates the complete specification. MPTA-Repair therefore collects the newly observed diagnostic trace and adds it to the pool for the next refinement round.

After full-property regression succeeds, MPTA-Repair applies the TARTAR-style admissibility test based on functional equivalence [11]. This check is needed because a clock-bound edit may satisfy safety properties by preventing intended behavior from occurring. TARTAR observes that timed-language equivalence is not a suitable admissibility criterion, because a successful timing repair is expected to remove at least some violating timed realizations [11]. Instead, functional equivalence compares the untimed languages of the original and repaired models [28]. Intuitively, the repaired model should not add or remove functional action traces, even though the timing of those traces may change.

In implementation, this criterion is checked by constructing untimed automata from the original and repaired timed models and comparing their accepted untimed languages [29]. If the languages differ, the repair is rejected and a separating behavior can be reported when available. Only candidates that pass both full-property regression and functional-equivalence admissibility are accepted.

The refinement loop therefore combines local repair generation with global validation. Local TDT encodings generate candidate bound assignments; full-property regression rejects candidates that do not satisfy the complete specification; admissibility rejects candidates that satisfy the properties by changing functional behavior; and blocking constraints prevent repeated failed assignments. The final result is a repair that satisfies the currently stated multi-property specification while preserving the intended untimed behavior of the model.

IV. Implementation and Experimental Evaluation

This section details the implementation of MPTA-Repair and evaluates its effectiveness across benchmark and industrial case studies. To maintain practical applicability, our methodology focuses on repairing numerical clock bounds within transition guards and location invariants, while avoiding arbitrary alterations to the discrete transition logic. This parametric repair space is pragmatic for industrial contexts, because it can correct clerical modeling mistakes and support redimensioning of system timing resources without disrupting the intended functional design. We further compare MPTA-Repair with an iterative baseline extended from the TARTAR paradigm [12]; the comparison illustrates why multi-property regression and multi-counterexample analysis are necessary to avoid secondary errors during repair.

A. Prototype Implementation

We implemented MPTA-Repair as an incremental repair pipeline that systematically processes a faulty system into a validated repair candidate or a failure report. The pipeline uses a timed-automata parser, a verification interface based on UPPAAL [19], a Z3-powered MaxSMT backend [26], and an admissibility checker. Inspired by a control-loop architecture, the pipeline computes repairs through the following sequential stages.

Input and initial verification. The pipeline ingests the faulty model, property set, and configuration parameters. It extracts editable clock-bound variables from transition guards and location invariants, then invokes full-property verification to generate an initial property-trace pool containing timed diagnostic traces for all violated properties.

Relation-guided search and optimization. The trace analyzer maps observed diagnostic traces to the extracted repair variables. A relation-analysis mechanism performs pruning, grouping, and ranking by establishing dependencies among properties, counterexamples, and repair sites. This mapping structures the repaired-bound search space and reduces the computational burden of candidate generation.

TABLE I
Statistics of the benchmark and industrial datasets.

Dataset	Original models	Mutated models	Avg. properties/model
Benchmark	9	54	4.1
Industrial	4	105	7.3

MaxSMT solving and candidate generation. The optimized repair evidence is encoded into a MaxSMT problem with hard constraints for trace feasibility, well-formedness, and blocking, together with soft constraints that prefer minimal modifications. Once the solver synthesizes a viable assignment, the model writer instantiates the corresponding candidate model.

Full validation and refinement. Each candidate is evaluated by two consecutive validation gates. The first gate performs full-property regression verification through UPPAAL to prevent collateral property violations. The second gate performs an acceptability check for untimed functional equivalence using a TARTAR-style criterion [11]; candidates that alter functional untimed behavior are rejected.

Iteration. If either validation gate rejects a candidate, the control loop feeds newly exposed diagnostic traces back into the trace pool. This initiates another counterexample-guided refinement iteration, which further constrains the solver with updated evidence until a globally admissible repair is found or the configured search budget is exhausted.

B. Evaluation Strategy and Datasets

To evaluate MPTA-Repair, we use two dataset groups designed to exercise different operational dimensions, with statistics summarized in Table I. The benchmark dataset contains nine established models selected from standard UPPAAL and XtaBenchmarkSuite sources, including examples such as Elevator and FDDI. These models provide structural diversity and support comparison with existing single-trace repair settings. The industry dataset contains four coupled industrial case studies: the Gear Control System for automotive transmission, SAE RoR for autonomous-driving decision making, Train Controller Alarm for railway emergency timing, and Train Gate Controller for shared-resource concurrency. Unlike many standard benchmarks, these industrial models contain dense multi-property timing constraints.

Because naturally occurring faulty timed automata are scarce, we synthesize faulty instances using a mutation-based evaluation strategy [24]. Fault injection strictly modifies numerical clock-bound constants in guards and invariants, deliberately excluding arbitrary structural logic changes. Simple mutants contain a localized single-point modification. Complex mutants use a two-fault effective mutation strategy that combines two non-redundant single-point faults. These combinations are filtered to retain only instances in which both faults contribute to the property violation. All retained mutants must remain syntactically valid, violate at least one

TABLE II
REPAIR SUCCESS RATES ACROSS BENCHMARK AND INDUSTRIAL DATASETS.

Dataset	Method	Success rate
Benchmark	Iterative baseline	72%
Benchmark	MPTA-Repair	93%
Industrial	Iterative baseline	20%
Industrial	MPTA-Repair	76%

selected timing safety specification, and pass a preliminary admissibility screen ensuring that the original non-timing functional behavior is preserved before repair.

We compare MPTA-Repair with an adapted iterative TARTAR-style baseline [12]. To ensure fairness, this baseline uses the same full-property regression and admissibility validation gates as MPTA-Repair. The key methodological difference is that the baseline derives repairs from only one property or diagnostic trace at a time. Because it lacks the joint multi-counterexample relation analysis used by MPTA-Repair, it is more prone to localized optimization and overfitting to isolated fault paths. The comparison therefore evaluates the benefit of treating properties, counterexamples, and repair variables as joint repair evidence.

C. Experimental Results

The experimental evaluation shows that MPTA-Repair outperforms the iterative baseline, particularly on complex industrial models. As shown in Table II, the baseline achieves a 72% success rate on the standardized benchmark dataset, whereas MPTA-Repair reaches 93%. The performance gap widens on the industry dataset, where the baseline repairs only 20% of the cases compared with 76% for MPTA-Repair. These results indicate that the proposed workflow remains effective on established timed-automata benchmarks while providing stronger robustness when properties and traces interact in dense industrial systems.

The baseline remains competitive on simpler benchmark models because timing errors are often concentrated on isolated guards or invariants. In such localized scenarios, a single diagnostic trace may provide enough evidence to infer a correct repair. However, even within standard benchmarks with shared multi-channel structures, such as FDDI protocol models, MPTA-Repair is more robust because it prevents single-trace overfitting. This advantage becomes clearer in industrial models with interacting concurrent components, urgent or broadcast synchronization channels, observer templates, and array-based modeling constructs. In these settings, localized baseline repairs frequently fail subsequent global validation checks. Our analysis indicates that the baseline lacks explicit optimization for edit magnitude, which can produce extreme candidate bounds that fail admissibility, and that it struggles with the regression pressure imposed by three to nine coupled domain properties without jointly analyzing their counterexamples.

MPTA-Repair succeeds more frequently due to two architectural advantages. First, multi-counterexample accumulation reduces overfitting to any single timed diagnostic trace, which typically exposes only one timing realization of a deeper defect. By accumulating and analyzing diagnostic traces from multiple operational modes, the framework synthesizes small joint edit sets that address the shared timing fault. Second, relation-guided optimization works with precise MaxSMT objectives to improve candidate quality. By identifying diagnostic traces that share path fragments or target repair variables, the solver focuses on numerical clock bounds close to the observed violations while penalizing large edit magnitudes. Successful repairs therefore typically modify only one or two critical clock bounds, which simplifies later manual inspection and verification by engineers.

D. Discussion and Practical Lessons

The evaluation highlights several lessons for applying automated repair to industrial timed automata. First, full-property regression testing is necessary. Because industrial system properties are highly interdependent, a localized fix designed only to satisfy a liveness-like response monitor can inadvertently violate a separate safety monitor that constrains maximum waiting time. MPTA-Repair avoids such behavioral regressions by treating restoration of the full property set as an indivisible global objective.

Second, admissibility changes the meaning of repair success. Without checking structural and functional equivalence, a solver could satisfy a timed property by disabling intended behavior, for example by preventing a critical system action. The admissibility check prevents property satisfaction from being achieved through unacceptable loss of functional untimed behavior.

Finally, failure analysis of unrepaired cases reveals limits of the current repair space. Computational timeouts and out-of-scope model syntax account for only part of the unsuccessful attempts. In highly coupled domains such as cardiac pacing, some errors require paired and consistent adjustments across both guards and invariants, which complicates the search space. Incomplete diagnostic trace coverage returned by the verification engine can also lead to residual overfitting. Most fundamentally, some complex cases lack a feasible candidate within the restricted clock-bound parameter space. This limitation suggests that future work should consider broader structural modifications, such as changing synchronization labels, adjusting clock resets, or redesigning discrete data updates.

V. CONCLUSION

This paper presented **MPTA-Repair**, an automatic repair workflow for multi-property, multi-counterexample clock-bound repair of industrial timed-automata models. The method addresses a practical limitation of local TDT-based repair: industrial models are commonly validated against multiple timing properties, and one timing defect may produce several related counterexamples. Repairing one trace in isolation may therefore be insufficient.

MPTA-Repair collects diagnostic traces from violated properties, uses lightweight relations among properties, traces, and repairable bounds to guide repair search, and synthesizes small clock-bound edits using a MaxSMT-based backend. Every candidate is validated by full-property regression and a TARTAR-style admissibility check based on functional equivalence [11]. This design ensures that accepted repairs are not merely local counterexample fixes, but system-level timing corrections that preserve intended behavior.

Our evaluation on benchmark and industry timed-automata datasets shows that MPTA-Repair improves repair success over an iterative TARTAR-style baseline, especially on industry cases where multi-property interaction and admissibility constraints are prominent. The results also show that failures remain meaningful: unrepaired cases often require changes outside the clock-bound repair space or exceed the configured solving and validation budgets.

Future work will extend the repair space beyond numerical clock bounds to include resets, clock references, synchronization labels, and structural edits. We also plan to strengthen counterexample generation, improve relation-guided decomposition, and evaluate the method on larger industrial timed-automata models.

REFERENCES

[1] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994.

[2] J. Bengtsson and W. Yi, "Timed automata: Semantics, algorithms and tools," in *Lectures on Concurrency and Petri Nets*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3098, pp. 87–124.

[3] D. Basile, A. Fantechi, and I. Rosadi, "Formal analysis of the UNISIG safety application intermediate sub-layer," in *Proceedings of Formal Methods for Industrial Critical Systems*, 2021, pp. 176–193.

[4] M. Lindahl, P. Pettersson, and W. Yi, "Formal design and analysis of a gear controller," in *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems*, 1998, pp. 281–297.

[5] A. Belguidoum, A. S. Al-Sibahi *et al.*, "A double-level model checking approach for an agent-based autonomous vehicle and road junction regulations," *Electronics*, vol. 10, no. 23, p. 3040, 2021.

[6] Z. Jiang, M. Pajic, R. Alur, and R. Mangharam, "Modeling and verification of a dual chamber implantable pacemaker," in *Proceedings of the IEEE Real-Time and Embedded Technology and Applications Symposium*, 2012, pp. 188–203.

[7] J. Cuartas *et al.*, "Formal verification of a mechanical ventilator using UPPAAL," in *Proceedings of Formal Techniques for Safety-Critical Systems*, 2023.

[8] A. David, K. G. Larsen, A. Legay, M. Mikučionis, and D. B. Poulsen, "Herschel-planck software schedulability analysis using UPPAAL," in *Proceedings of Leveraging Applications of Formal Methods, Verification and Validation*, 2010, pp. 282–296.

[9] T. Vogel, M. Carwehl, G. N. Rodrigues, and L. Grunske, "A property specification pattern catalog for real-time system verification with UPPAAL," 2022, arXiv:2211.03817.

[10] C. Daws, A. Olivero, S. Tripakis, and S. Yovine, "The tool KRONOS," in *Hybrid Systems III*, ser. Lecture Notes in Computer Science. Springer, 1996, vol. 1066, pp. 208–219.

[11] M. Koelbl, S. Leue, and T. Wies, "Clock bound repair for timed systems," 2019, manuscript.

[12] ——, "TarTar: A timed automata repair tool," 2020, arXiv:2002.02760.

[13] R. Alur, T. A. Henzinger, and M. Y. Vardi, "Parametric real-time reasoning," in *Proceedings of the ACM Symposium on Theory of Computing*, 1993, pp. 592–601.

[14] Étienne André, P. Arcaini, A. Gargantini, and M. Radavelli, "Repairing timed automata clock guards through abstraction and testing," in *Proceedings of Tests and Proofs*, 2019.

[15] J. Bendík, A. Sencan, E. A. Gol, and I. Černá, "Timed automata robustness analysis via model checking," 2021, arXiv:2108.08018.

[16] M. Jose and R. Majumdar, "Bug-assist: Assisting fault localization in ANSI-C programs," in *Proceedings of Computer Aided Verification*, 2011, pp. 504–509.

[17] A. González, M. Cristiá, and C. Luna, "Verification of quantitative temporal properties in RealTime-DEVS," 2024, arXiv:2409.18732.

[18] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on Uppaal," in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science, vol. 3185. Springer, 2004, pp. 200–236.

[19] K. G. Larsen, P. Pettersson, and W. Yi, "UPPAAL in a nutshell," *International Journal on Software Tools for Technology Transfer*, vol. 1, no. 1–2, pp. 134–152, 1997.

[20] P. Han, Z. Zhai, B. Nielsen, and U. Nyman, "A modeling framework for schedulability analysis of distributed avionics systems," 2018, arXiv:1803.11050.

[21] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.

[22] R. Sebastiani and S. Tomasi, "Optimization modulo theories with linear rational costs," *ACM Transactions on Computational Logic*, vol. 16, no. 2, 2015.

[23] E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-guided abstraction refinement," in *Proceedings of Computer Aided Verification*, 2000, pp. 154–169.

[24] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[25] R. Sebastiani and P. Trentin, "On optimization modulo theories, MaxSMT and sorting networks," 2017, arXiv:1702.02385.

[26] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proceedings of Tools and Algorithms for the Construction and Analysis of Systems*, 2008, pp. 337–340.

[27] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. MIT Press, 1999.

[28] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 2nd ed. Addison-Wesley, 2001.

[29] P. Bouyer, P. Gastin, F. Herbreteau, O. Sankur, and B. Srivathsan, "Zone-based verification of timed automata: Extrapolations, simulations and what next?" 2022, arXiv:2207.07479.